

When Reflection Helps: Utility-Guided Advantage Rescaling for Efficient Reasoning

Anonymous ACL submission

Abstract

Large reasoning models have recently demonstrated emergent “aha moments” that enable reflective reasoning. However, a non-trivial portion of reflective steps are “low-utility,” where reasoning errors are not corrected, or even correct reasoning trajectories are shifted toward incorrect ones. To address low-utility reflective continuation, this paper proposes **RUAR**, a utility-guided advantage rescaling method for reflective reasoning. During reinforcement learning (RL), RUAR rescales the token-level advantages within each reflective reasoning step according to the estimated utility of the reflective continuation. The utility is estimated through **answer-forcing utility estimation**, which measures how the model’s correct-answer probability changes before and after the reflective step by forcing answer generation from both reasoning states using answer-forcing prompts. Furthermore, RUAR introduces an **overthinking-mitigating rescaling mechanism**, where utility-guided rescaling is no longer applied from the earliest answer-ready reflection step onward. Experiments demonstrate that RUAR improves the accuracy–efficiency trade-off, substantially reducing generated token usage while often preserving or improving final-answer accuracy.

1 Introduction

Recently, large reasoning models (LRMs) have demonstrated strong performance on complex reasoning tasks by leveraging increased inference-time computation through extended intermediate reasoning (Snell et al., 2025; Guo et al., 2025). A key factor underlying these capabilities is the emergence of “aha moments,” a form of reflective reasoning in which models revisit, verify, and reorganize an existing reasoning trajectory, thereby recovering from earlier errors and eventually reaching correct solutions to challenging problems through iterative reflection (Guo et al., 2025).

Are Reflective Tokens Always Useful?

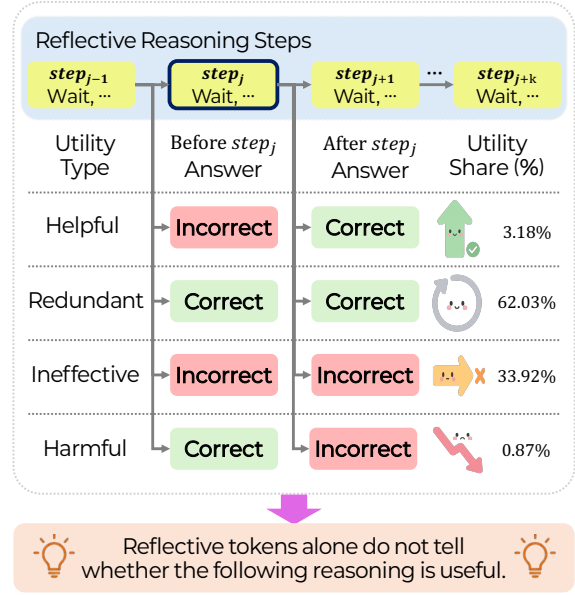


Figure 1: Utility distribution over 8,590 extracted *Wait* reflective reasoning steps from Qwen3-8B MATH500 base-model traces. For each step, answer-forcing generates final answers from the before-step and after-step partial traces, and each answer is compared with the gold answer to identify whether the step is *helpful*, *redundant*, *ineffective*, or *harmful*.

Such reflective behavior is often associated with reflective tokens such as *Wait* and *Alternatively*, which allow models to revisit and extend an existing reasoning trajectory (Guo et al., 2025). However, contrary to the expected self-correction effect of reflective continuation, reflection is not always useful (Huang et al., 2024; Wang et al., 2025). As illustrated in Figure 1, only 3.18% of extracted *Wait* steps are helpful, while 96.82% are redundant, ineffective, or harmful. We refer to these latter three categories as **low-utility** reflection: continuation that consumes additional reasoning tokens without improving the probability of producing the correct answer. Thus, reflective tokens provide use-

ful boundaries for reflective reasoning steps, but they are not reliable utility signals.

Prior work has mainly handled low-utility reflective continuation by reducing or stopping reflection at a coarser level. Some methods suppress reflective tokens during decoding to disable explicit self-reflection (Wang et al., 2025). Others observe that later reflection, especially after a candidate answer has appeared, is often confirmatory, and therefore truncate subsequent reflection or stop reasoning early (Kang et al., 2026). More general efficient-reasoning methods prune redundant reasoning chunks using attention (Choi et al., 2025) or verifier signals (Lin et al., 2026), or train models with length-oriented objectives (Luo et al., 2025) and token-budget constraints (Hou et al., 2026). These approaches can reduce token usage, but they do not directly determine whether a particular reflective reasoning step is helpful, redundant, ineffective, or harmful.

Motivated by the need to suppress low-utility reflection while preserving corrective high-utility reflection, we propose RUAR, *reflective reasoning with utility-guided advantage rescaling*. Instead of using a fixed trace-level advantage score uniformly across all tokens in a reasoning trace during reinforcement learning (RL) training, RUAR rescales token-level advantage signals for each reflective reasoning step according to its estimated utility. In particular, RUAR performs **answer-forcing utility estimation**: for each reflective reasoning step, RUAR applies *answer-forcing prompts* to both the “before-step” and “after-step” partial traces, compares the resulting forced-answer correctness rates, and estimates how the correct-answer probability changes before and after the reflective continuation.

Furthermore, once the model reaches the earliest answer-ready reflection step, additional reflective continuation can become redundant and contribute to overthinking behavior. To mitigate this issue, RUAR further introduces an **overthinking-mitigating rescaling method**, where utility-guided rescaling is no longer applied from the earliest answer-ready reflection step onward, and a fixed sign-dependent rescaling strategy is instead employed depending on whether the original advantage is positive or negative. As a result, RUAR enables step-level credit assignment without requiring human process labels, learned process reward models, or step-level annotations.

Our contributions are threefold. First, we empirically show that reflective reasoning steps dif-

fer substantially in their self-correction effectiveness, and that a non-trivial portion of reflective continuation suffers from low-utility behavior. Second, we propose RUAR, a utility-guided advantage rescaling method for reflective reasoning, consisting of answer-forcing-driven utility estimation and an overthinking-mitigating rescaling mechanism. Third, experiments on open-weight reasoning models demonstrate that RUAR improves the accuracy–efficiency trade-off, substantially reducing generated token usage while often preserving or improving final-answer accuracy.

2 Preliminaries

2.1 Reasoning by Multiple Reflection Steps

Given a question (or prompt) x , the policy of an LRM, denoted by π_θ , generates a full response $\mathbf{y} = (y_1, \dots, y_T)$, where T is the total number of generated tokens. The full response is decomposed into a reasoning segment and an answer segment, written as $\mathbf{y} = [\mathbf{y}_r; \mathbf{y}_a]$, where $\mathbf{y}_r = (y_1, \dots, y_{T_r})$ denotes the reasoning segment and $\mathbf{y}_a = (y_{T_r+1}, \dots, y_T)$ denotes the answer segment. The two segments are separated by the closing thinking tag $\langle \text{\/think} \rangle$.

During reflective reasoning, the reasoning segment $\mathbf{y}_r$ exhibits an extended thinking process through the frequent use of reflective tokens such as *Wait* and *Alternatively*. By treating a subset of reflective tokens $\mathcal{R}$ as target step-start delimiters while including the corresponding reflection token itself within each reflective step, we extract a sequence of reflective reasoning steps $(s_1, \dots, s_m)$ from $\mathbf{y}_r$, where $s_j = y_{a_j}, \dots, y_{e_j}$ denotes the j -th reflective reasoning step, spanning from the a_j -th token to the e_j -th token in $\mathbf{y}$. These extracted reflective reasoning steps need not cover the entire reasoning segment. The starting token of each reflective step, $y_{a_j} \in \mathcal{R}$, is always the reflective token used as the target step-start delimiter. In this paper, for simplicity, we use $\mathcal{R} = \{\text{“Wait”}\}$ as target step-start delimiters, while both *Wait* and *Alternatively* are used only to determine step endpoints. If no later endpoint delimiter exists, the final reflective step ends before the closing thinking tag $\langle \text{\/think} \rangle$. Appendix H provides a method-aligned inventory of these reflective tokens, and Appendix N provides a concrete example of this endpoint-delimitation rule.

As discussed in Section 1, there is no guarantee that reflective reasoning steps consistently improve

answer correctness throughout the reasoning trajectory. This motivates estimating the utility of each reflective step based on its before/after effect on correct-answer probability, rather than assigning static rewards or penalties solely based on the presence of reflective tokens.

2.2 Length-Regularized RLOO Trainer

For each prompt x , we sample N rollouts $\{\mathbf{y}^{(i)}\}_{i=1}^N$ from the current policy and assign each rollout a terminal reward R_i . Our base trainer uses a REINFORCE leave-one-out (RLOO) advantage estimator (Ahmadian et al., 2024). For rollout i , the leave-one-out baseline and rollout-level advantage are

$$\bar{R}_{-i} = \frac{1}{N-1} \sum_{n \neq i} R_n, \quad \hat{A}_i = R_i - \bar{R}_{-i}.$$

This rollout-level signal is broadcast to response tokens, yielding the base token-level advantage signal $A_{i,t}$ that RUAR later rescales before the final policy update.

Following prior efficient-reasoning work (Arora and Zanette, 2026), we use a length-regularized terminal reward in the RLOO trainer. The exact reward definition is provided in Appendix F.

3 Method

3.1 Overview

As discussed in Section 2.2, standard outcome-supervised RL assigns the same trace-level supervision signal to a full response by using identical advantage scores across all tokens in a reasoning trace, without considering that reflective reasoning steps may differ substantially in whether they correct, repeat, or derail the solution trajectory. Instead of using trace-constant advantages $A_{i,t}$, RUAR estimates the utility of each reflective reasoning step s_j from its before/after effect on correct-answer probability under answer-forcing prompts, and uses this utility to reshape token-level advantage signals during RL training.

In addition, to mitigate overthinking, RUAR further identifies the *earliest answer-ready step*: before this step, it applies utility-guided rescaling to reflective reasoning steps, while after this step, it applies fixed overthinking-mitigating rescaling to redundant post-ready continuation. For readability, we use simplified notation for step-level quantities within a fixed rollout, while Algorithm 1 uses rollout-indexed notation for the full training procedure.

3.2 Answer-Forcing Utility Estimation

With multiple reflective reasoning steps defined in Section 2.1, RUAR performs answer-forcing utility estimation to determine whether each reflective step improves correct-answer probability. For the j -th reflective reasoning step s_j in $\mathbf{y}$, defined as $s_j = y_{a_j}, \dots, y_{e_j}$, we define the corresponding *before-step* and *after-step* partial traces as

$$\mathbf{s}_j^- = \mathbf{y}_{1:a_j-1}, \quad \mathbf{s}_j^+ = \mathbf{y}_{1:e_j}. \quad (1)$$

Here, $\mathbf{y}_{m:n}$ denotes the span of response tokens starting from y_m and ending at y_n .

Given a before-step or after-step partial trace $\mathbf{y}_{1:q}$ (i.e., either $\mathbf{s}_j^-$ or $\mathbf{s}_j^+$), we apply an *answer-forcing prompt* $\mathcal{A}$ to evaluate whether $\mathbf{y}_{1:q}$ contains sufficient reasoning information for the model π_θ to generate the correct final answer when explicitly forced to answer, as follows:

\n**Final Answer**\n\boxed

We then sample K short *final-answer continuations* from the current policy. Let $o_{j,k}^-$ and $o_{j,k}^+$ denote the k -th continuations sampled from $\mathbf{s}_j^- \oplus \mathcal{A}$ and $\mathbf{s}_j^+ \oplus \mathcal{A}$, respectively, where $\oplus$ denotes string concatenation. Let `correct` be a *rule-based final-answer verifier* satisfying `correct`($\cdot$) $\in \{0, 1\}$. The estimated correct-answer probabilities for the before-step and after-step partial traces are computed as follows:

$$p_j^- = \frac{1}{K} \sum_{k=1}^K \text{correct}(x, \mathbf{s}_j^- \oplus \mathcal{A} \oplus o_{j,k}^-), \quad (2)$$

$$p_j^+ = \frac{1}{K} \sum_{k=1}^K \text{correct}(x, \mathbf{s}_j^+ \oplus \mathcal{A} \oplus o_{j,k}^+).$$

where p_j^- and p_j^+ denote the *before-step* and *after-step correct-answer probabilities*, respectively.

The utility of the reflective reasoning step s_j is then defined as

$$u_j = p_j^+ - p_j^-. \quad (3)$$

The verifier `correct` evaluates only the sampled final-answer continuations and does not provide human process labels or step-level supervision. See Appendix G for implementation details of the answer-forcing prompt and verification procedure.

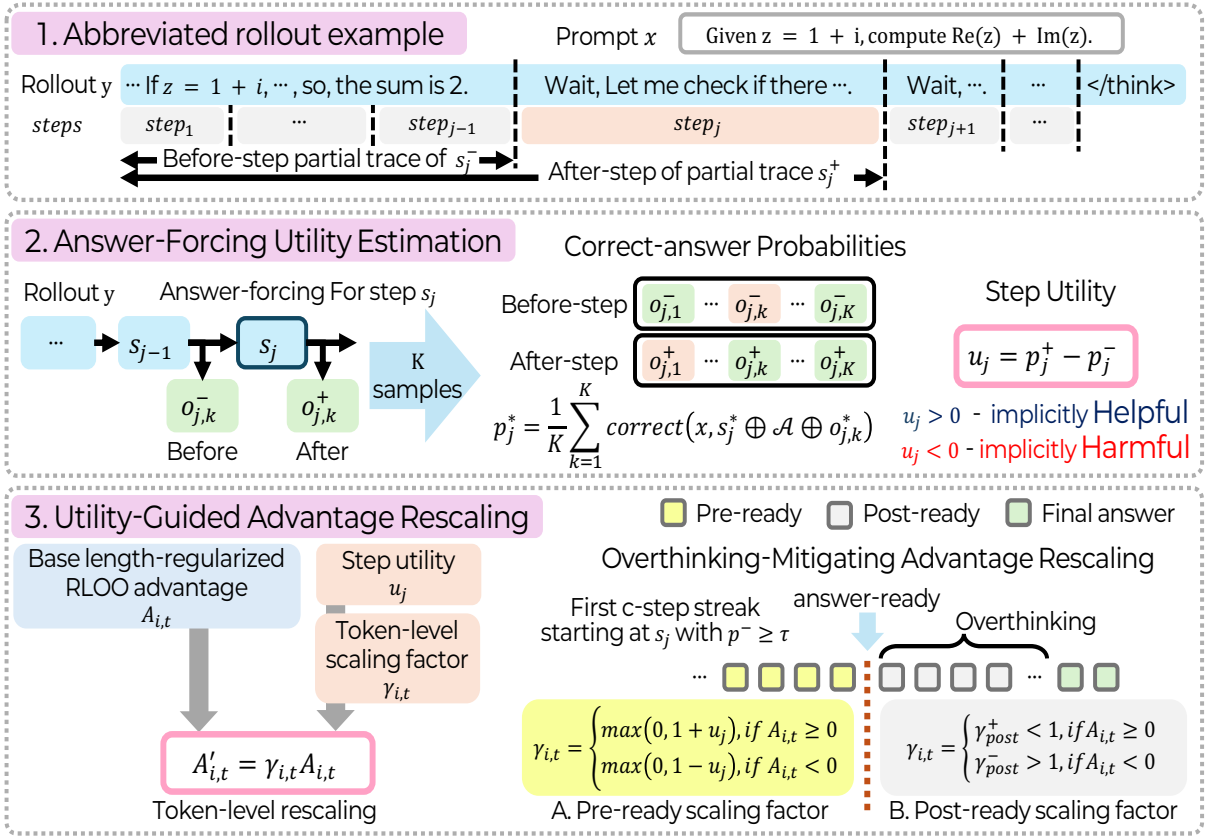


Figure 2: An overall pipeline of RUAR on an abbreviated rollout. [1] Given a full rollout y , RUAR first segments y into m reflective reasoning steps, $(s_j)_{j=1}^m$. The *before-step* and *after-step* partial traces of s_j correspond to the partial traces immediately before and after s_j , respectively, as defined in Eq. 1. [2] Answer-forcing utility estimation appends the answer-forcing prompt $\mathcal{A}$ to both the before-step and after-step partial traces, i.e., s_j^- and s_j^+ , and immediately generates a set of answer completions, denoted by $\{o_{j,k}^-\}_{k=1}^K$ and $\{o_{j,k}^+\}_{k=1}^K$, respectively. The corresponding correct-answer probabilities, denoted by p_j^- and p_j^+ and computed using Eq. 2, are then used to estimate the utility of s_j , where $u_j = p_j^+ - p_j^-$. [3] To mitigate overthinking behavior, utility-guided advantage rescaling is applied only to pre-ready reflective reasoning steps, where the model is not yet sufficiently ready to generate the correct answer. For the remaining post-ready reflective reasoning steps, a simpler constant-based scaling strategy is applied, reducing the likelihood of generating additional reflective continuation, as defined in Eq. 7.

3.3 Utility-Guided Advantage Rescaling

Let $A_{i,t}$ denote the base token-level advantage signal assigned by the length-regularized RLOO trainer before the final policy update. RUAR constructs a token-level scaling factor $\gamma_{i,t}$ and computes the rescaled advantage signal as follows:

$$A'_{i,t} = \gamma_{i,t} A_{i,t}. \quad (4)$$

The RL trainer subsequently applies its standard advantage normalization procedure to $A'_{i,t}$ before the PPO-style policy update.

For a token t belonging to a reflective reasoning step s_j to which utility-guided rescaling is applied, RUAR computes the scaling factor based on the

estimated utility u_j of the reflective step:

$$\gamma_{i,t} = \begin{cases} \max(0, 1 + u_j), & A_{i,t} \geq 0, \\ \max(0, 1 - u_j), & A_{i,t} < 0. \end{cases} \quad (5)$$

Thus, when $A_{i,t}$ is positive, helpful reflective steps receive stronger positive credit, while harmful reflective steps receive weaker positive credit. Conversely, when $A_{i,t}$ is negative, helpful reflective steps are penalized less, whereas harmful reflective steps are penalized more strongly.

3.4 Overthinking-Mitigating Rescaling

The utility-guided rescaling rule in Section 3.3 assigns step-specific credit according to estimated

step utility. However, once a reasoning trajectory is already likely to produce the correct final answer, additional reflective continuation can become redundant. RUAR therefore applies utility-guided rescaling only before the model becomes answer-ready, and switches to fixed post-ready rescaling afterward.

3.4.1 Pre-Ready and Post-Ready Reflection

We divide reflective reasoning into two stages: pre-ready and post-ready.

1) **Pre-ready stage.** At the pre-ready stage, the current reasoning trajectory is not yet sufficiently reliable, and further exploration or self-correction may still be required before the model becomes ready to generate the correct answer.

2) **Post-ready stage.** At the post-ready stage, the reasoning trajectory has become sufficiently reliable, and the model is already likely to generate the correct answer. Additional reflective continuation after this point is therefore more likely to become redundant and contribute to overthinking behavior.

To identify the transition between these stages, we define a reflective step as *near answer-ready* if its before-step correct-answer probability is at least a threshold τ . If the j -th reflective step and its following $c - 1$ consecutive reflective steps are all near answer-ready, then the j -th step is identified as the earliest answer-ready reflection step, referred to simply as the *answer-ready step*. Formally, the index b of the answer-ready step is

$$b = \min\{j : p_r^- \geq \tau \text{ for } r = j, \dots, j + c - 1\}. \quad (6)$$

If no such consecutive run exists, we set $b = m + 1$.

Using the answer-ready step s_b , we define pre-ready and post-ready reflective reasoning steps as follows: 1) **Pre-ready reflective steps:** $\mathbf{y}^{\text{pre}} = (s_1, \dots, s_{b-1})$. 2) **Post-ready reflective steps:** $\mathbf{y}^{\text{post}} = (s_b, \dots, s_m)$.

Here, $\mathbf{y}^{\text{pre}}$ contains reflective steps before the answer-ready step, while $\mathbf{y}^{\text{post}}$ contains the answer-ready step itself and later reflective continuation, i.e., from the answer-ready step onward, inclusive. During training, once the answer-ready step is detected, RUAR stops computing additional answer-forcing utility estimates for the remaining reflective continuation within the rollout. A concrete probing example is provided in Appendix O, while Appendix I further analyzes the behavior of answer-ready step detection.

3.4.2 Pre-Ready and Post-Ready Rescaling

Using the pre-ready and post-ready reflective reasoning steps defined above, RUAR specifies which scaling factor $\gamma_{i,t}$ is used in Eq. 4 for each token region. For pre-ready tokens $y_t \in \mathbf{y}^{\text{pre}}$, the scaling factor $\gamma_{i,t}$ follows the utility-guided rule in Section 3.3. For post-ready continuation, RUAR no longer uses the estimated step utility. Once the answer-ready step s_b is detected, fixed post-ready rescaling is applied to all remaining reasoning-segment tokens from the start of s_b onward:

$$\gamma_{i,t} = \begin{cases} \gamma_{\text{post}}^+, & A_{i,t} \geq 0, \\ \gamma_{\text{post}}^-, & A_{i,t} < 0, \end{cases} \quad (7)$$

where $\gamma_{\text{post}}^+ < 1$ and $\gamma_{\text{post}}^- > 1$.

Tokens outside both pre-ready reflective reasoning steps and this post-ready continuation use the default scaling factor $\gamma_{i,t} = 1$.¹

3.5 Policy Optimization

After standard advantage normalization, we use the normalized RUAR-adjusted advantage signal in the same PPO-style policy objective as the base trainer:

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}_{i,t} [\min(r_{i,t} A'_{i,t}, \text{clip}_{\epsilon}(r_{i,t}) A'_{i,t})]. \quad (8)$$

where

$$\text{clip}_{\epsilon}(r) = \text{clip}(r, 1 - \epsilon, 1 + \epsilon). \quad (9)$$

Here, $r_{i,t}$ denotes the policy likelihood ratio. RUAR preserves the original outcome-based RL trainer, including rollout generation, terminal outcome scoring, and terminal reward computation. The additional component introduced by RUAR is a step-level rescaling layer that applies utility-guided scaling before the answer-ready step and fixed post-ready scaling afterward.

4 Experiments

4.1 Experimental Setup

Backbones and baselines. We evaluate RUAR as a post-training method for improving the

¹In the main setting, *Alternatively*-start continuations before the answer-ready step are not selected as target reflective reasoning steps for utility-guided rescaling, although *Alternatively* can terminate a preceding *Wait*-start step. See Appendix N for a concrete example.

Methods	GSM8K			MATH500			AIME2024			HMMT2025			GPQA			ARC-C			CSQA		
	Acc.	Tok.	AES	Acc.	Tok.	AES	Acc.	Tok.	AES	Acc.	Tok.	AES	Acc.	Tok.	AES	Acc.	Tok.	AES	Acc.	Tok.	AES
DeepSeek-R1-Distill-Qwen-1.5B																					
Base	75.7	956	0.00	67.6	3519	0.00	13.3	9009	0.00	6.7	8804	0.00	15.2	5318	0.00	56.2	1046	0.00	43.5	810	0.00
PRIME	77.3	676	0.36	72.6	4428	-0.04	16.7	13483	0.27	10.0	16440	0.61	31.3	6846	2.89	57.0	959	0.13	41.8	810	-0.20
TLMRE	83.4	522	0.76	79.2	1784	1.01	16.7	6698	1.02	6.7	7635	0.13	19.7	3821	1.17	57.5	655	0.44	45.9	508	0.54
RUAR	83.5	523	0.76	80.0	1567	1.10	20.0	6427	1.80	13.3	4928	3.40	30.8	2673	3.58	62.6	386	0.97	47.9	324	0.90
Qwen3-8B																					
Base	95.1	2153	0.00	93.2	5343	0.00	73.3	13668	0.00	36.7	18920	0.00	57.9	8685	0.00	93.9	1340	0.00	83.9	1339	0.00
PRIME	92.7	4150	-1.05	86.6	10184	-1.26	56.7	22510	-1.78	26.7	26135	-1.74	54.3	18551	-1.44	94.3	3574	-1.65	81.9	6471	-3.96
TLMRE	95.6	1012	0.54	92.8	3414	0.34	46.7	15375	-1.94	40.0	14485	0.51	54.3	8456	-0.28	94.1	909	0.33	83.7	1089	0.17
RUAR	96.0	865	0.62	92.8	2564	0.50	73.3	9444	0.31	33.3	14015	-0.20	55.8	5255	0.22	94.5	738	0.47	84.7	729	0.48

Table 1: Acc. is reported in percent; Tok. is mean response length in tokens; AES is computed relative to Base within each backbone and benchmark.

accuracy–efficiency trade-off of reasoning models. Our main comparison spans two open-weight reasoning backbones: DeepSeek-R1-Distill-Qwen-1.5B (Guo et al., 2025) and Qwen3-8B (Yang et al., 2025). For each backbone, we compare four policies: the original Base model, the length-regularized RL baseline of Arora and Zanette (2026), denoted TLMRE, PRIME (Cui et al., 2026), and RUAR. TLMRE serves as the direct length-efficiency baseline for RUAR, while PRIME is included as a process-signal baseline that, like RUAR, avoids human process labels but uses implicit reward modeling rather than reflective-step utility rescaling.

Training and checkpoint selection. The RUAR policies are trained with online outcome-supervised RL on Numina-3K, a subset of NuminaMath-CoT (Li et al., 2024). NuminaMath-CoT contains chain-of-thought math problem-solution pairs from diverse sources, including MATH, Olympiad problems, and AMC/AIME. For each prompt, we sample 16 rollouts and compute base advantages using the RLOO estimator (Ahmadian et al., 2024) before applying RUAR’s step-level rescaling. We train DeepSeek-R1-Distill-Qwen-1.5B for 150 steps and Qwen3-8B for 50 steps, and select checkpoints by the validation accuracy–length trade-off. For a fair comparison under the same training-step budget, TLMRE and PRIME are reported at the same training step as the selected RUAR checkpoint for each backbone. Full training budgets and baseline matching details are provided in Appendix B, with additional training dynamics and cost analyses in Appendix E.

Benchmarks. Our main table reports results on mathematical reasoning and held-out multiple-choice reasoning benchmarks. The mathematical group contains GSM8K (Cobbe et al., 2021),

MATH500 (Lightman et al., 2024), AIME2024 (Jia, 2024), and HMMT2025 (MathArena, 2025). Since training uses Numina-3K, we also evaluate out-of-domain generalization on non-math benchmarks: GPQA-Diamond (Rein et al., 2023), ARC-Challenge (Clark et al., 2018), and CommonsenseQA (Talmor et al., 2019). We additionally verify that the Numina-3K training split has no direct problem-statement overlap with the main evaluation benchmarks; details are provided in Appendix D.

Decoding and metrics. To isolate policy differences without adding sampling variance, all main-table results use greedy single-sample decoding for all methods, with one response per problem and a maximum response length of 32768 tokens. We report accuracy, average generated response length in tokens, and the accuracy-efficiency score (AES) from O1-Pruner (Luo et al., 2025), computed relative to the Base row. Full decoding, scoring, and AES computation details are provided in Appendix C.

4.2 Main Result

Table 1 shows that RUAR improves the accuracy–efficiency trade-off in most settings across model scales and benchmark types. Across two backbones and seven benchmarks, RUAR achieves the best or tied-best AES in 13 out of 14 settings and the shortest average response length in 13 out of 14 settings.

On mathematical reasoning benchmarks, RUAR substantially reduces generated tokens while often preserving or improving accuracy. For the 1.5B model, it improves accuracy across all math datasets while producing much shorter responses. For Qwen3-8B, it reduces tokens by up to roughly 60%, matching or improving Base accuracy on

GSM8K and AIME2024, with small drops on MATH500 and HMMT2025.

The trade-off gains also transfer to held-out non-math benchmarks: RUAR improves non-math accuracy for the 1.5B model and produces substantially shorter responses for both backbones. Compared with TLMRE, these results suggest benefits beyond length-only compression; compared with PRIME, which applies implicit token-level process rewards across the entire response, RUAR’s selective utility-based rescaling over reflective reasoning steps yields a stronger overall accuracy–efficiency trade-off.

4.3 Analysis

We further examine the gains in Table 1, asking whether RUAR merely shortens reflection or selectively reduces low-utility and post-ready continuation. The utility categories used below are diagnostic labels for analysis, whereas RUAR uses the utility estimate $u_j = p_j^+ - p_j^-$ for advantage rescaling during training.

Low-utility reflection after training. Table 2 applies the same answer-forcing utility diagnostic as Figure 1 to Qwen3-8B MATH500 traces from the evaluated policies. Compared with the base model, RUAR produces much shorter responses and substantially fewer extracted reflective reasoning steps, while the helpful-step share nearly doubles from 3.18% to 6.14% (+93.1% relative). More importantly, compared with the length-only TLMRE baseline, RUAR produces shorter responses and fewer steps, but retains more helpful reflection: the number of helpful steps increases from 146 to 158, and the helpful-step share increases from 4.07% to 6.14% (+50.9% relative). These results suggest that RUAR does not uniformly suppress reflection; instead, it shifts the policy toward denser high-utility reflection while reducing repeated low-utility continuation. Full utility-category details, including redundant, ineffective, harmful, and *Alternatively* token counts, are provided in Appendix L.

Post-ready continuation reduction. Table 3 decomposes the reasoning segment y_r defined in Section 2.1 into pre-ready and post-ready regions using the detected answer-ready step. The base model and PRIME spend a large fraction of their reasoning budget after becoming answer-ready, indicating substantial redundant continuation. TLMRE reduces post-ready continuation through length-oriented training, but RUAR further reduces it,

Method	Avg. Tok.	#Steps	Helpful	Low-util.
Base	5343	8590	273 (3.18)	8317 (96.82)
PRIME	10184	11380	333 (2.93)	11047 (97.07)
TLMRE	3414	3586	146 (4.07)	3440 (95.93)
RUAR	2564	2572	158 (6.14)	2414 (93.86)

Table 2: Utility composition of extracted target reflective reasoning steps on Qwen3-8B MATH500. Percentages are computed over extracted target reflective reasoning steps for each method. Low-util. combines redundant, ineffective, and harmful steps.

producing the fewest post-ready tokens and the lowest post-ready share among the compared methods. This empirically supports the motivation of overthinking-mitigating rescaling: RUAR’s token reduction is concentrated in the continuation regime after the model has already become answer-ready, rather than arising from uniform shortening alone. Appendix M confirms that post-ready reflective steps are mostly low-utility, and Appendix P provides a qualitative example.

Method	Pre tok.	Post tok.	Post %
Base	2215	2480	53
PRIME	2565	6893	73
TLMRE	2193	727	25
RUAR	1718	390	19

Table 3: Pre/post-ready token decomposition on Qwen3-8B MATH500. Pre tok. and Post tok. are average token counts over y_r , excluding y_a ; hence their sum does not equal the mean response length in Table 1.

4.4 Ablation Study

Overthinking-mitigating rescaling ablation. Table 4 ablates answer-ready step detection and fixed post-ready scaling. The w/o post-ready scaling variant keeps the answer-ready criterion but removes fixed post-ready rescaling by setting $\gamma_{\text{post}}^+ = \gamma_{\text{post}}^- = 1$, leading to substantially longer responses and lower accuracy. The w/o answer-ready step variant applies utility-guided rescaling to all selected reflective steps; although responses remain short, accuracy drops because the distinction between potentially useful pre-ready reflection and redundant post-ready continuation is removed. Full RUAR combines these two components: the answer-ready step identifies where fine-grained utility estimation should stop, while fixed post-ready scaling suppresses further continuation. Its average scaling factors are therefore shifted in the intended direction, reducing positive credit and in-

creasing negative penalties. Additional component and sensitivity analyses are provided in Appendices J and K.

Variant	Acc.	Avg. Tok.	#Wait	#Alt.	$\bar{\gamma}^+$	$\bar{\gamma}^-$
w/o post-ready scaling	75.2	2724	2282	806	1.012	0.988
w/o answer-ready step	75.4	2201	2220	564	1.012	0.988
Full RUAR	79.2	2222	1537	1202	0.926	1.018

Table 4: Overthinking-mitigating rescaling ablation on MATH500 using the 1.5B step-50 checkpoint. $\bar{\gamma}^+$ and $\bar{\gamma}^-$ denote average scaling factors for positive- and negative-advantage tokens.

5 Related Work

Efficient reasoning in large reasoning models.

Long chain-of-thought reasoning improves test-time problem solving but increases generation cost (Snell et al., 2025; Feng et al., 2025). Existing work therefore reduces the amount, representation, or timing of reasoning. Some methods compress explicit traces through controllable CoT compression, redundant-token pruning, or sketch-style reasoning (Xia et al., 2025; Choi et al., 2025; Aytes et al., 2025). Others restructure the form of reasoning through soft or latent representations (Xu et al., 2025; Zhang et al., 2026; Shi et al., 2026). A third direction adaptively controls reasoning effort by deciding when to think, continue, or exit (Zhang et al., 2025b; Xiang et al., 2026; Zhao et al., 2026; Shen et al., 2025; Lin et al., 2026; Yang et al., 2026). Training-based variants also add length-, difficulty-, or budget-aware rewards to favor concise correct answers (Arora and Zanette, 2026; Hou et al., 2026; Chen et al., 2026; Cheng et al., 2026; Liu et al., 2026). These methods mainly control how much reasoning is generated, represented, or retained. RUAR instead estimates which reflective reasoning steps are useful and uses this utility to rescale advantage signals during training.

Reflective tokens and self-correction. Reflection has been used for iterative feedback, verbal memory, tool-based critique, and deliberative search over reasoning paths (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024; Yao et al., 2023). However, self-correction is not uniformly reliable: models can fail to correct errors without external feedback, depend on strong verifiers, misidentify fallacious reasoning, or misjudge their own outputs (Huang et al., 2024; Zhang et al., 2024; Hong et al., 2024; Jiang et al., 2024). Recent work further studies reflection inside long reasoning

traces, including verbalized confidence, hidden self-verification, spontaneous correction, post-answer reflection, overused self-verification, reflection-token scheduling, and discourse-marker segmentation (Zeng et al., 2025; Zhang et al., 2025a; Zhao et al., 2025; Kang et al., 2026; Long et al., 2026; Fan et al., 2026; Wang et al., 2025). These findings support our view that reflective tokens such as *Wait* provide useful boundaries for reflective reasoning steps, but are not reliable utility signals.

Process-level signals without process labels.

Outcome-based RL trains reasoning models from terminal correctness using policy-gradient objectives, including PPO/REINFORCE-style optimization and recent value-free, group-relative, or sequence-level variants such as RLOO, GRPO, DAPO, and GSPO (Schulman et al., 2017; Ahmadian et al., 2024; Shao et al., 2024; Yu et al., 2026; Zheng et al., 2025a). Yet terminal rewards assign broad credit to whole rollouts and do not identify which intermediate reasoning steps helped or hurt. Process supervision addresses this with human process labels, automatic step-level labels, process reward models, or process-error benchmarks (Lightman et al., 2024; Wang et al., 2024; Zheng et al., 2025b). Other work reduces explicit process annotation by deriving implicit or learned process rewards from outcome supervision (Yuan et al., 2025; Cui et al., 2026). RUAR shares the goal of obtaining finer-grained process-level signals, but does not train a separate process reward model. Instead, it estimates before/after utility for reflective reasoning steps and converts that utility directly into token-level advantage rescaling.

6 Conclusion

We introduced RUAR, a utility-guided advantage rescaling method for reflective reasoning. RUAR estimates each reflective step’s before/after utility with answer-forcing probes and rescales token-level advantages without human process labels or a learned process reward model. It also mitigates overthinking by switching to fixed post-ready rescaling after the earliest answer-ready step. Experiments show that RUAR improves the accuracy–efficiency trade-off across mathematical and held-out non-math benchmarks, suggesting that efficient reasoning requires crediting which reflective steps are useful, not only how much the model thinks.

7 Limitations

This work intentionally focuses on *Wait-start* reflective reasoning steps as the main unit for utility-guided credit assignment. This choice gives a simple, interpretable, and computationally practical step definition, since probing broader reflective-token sets and alternative step definitions would substantially increase RL post-training cost. However, it also leaves richer forms of reflective continuation underexplored, including other discourse markers, model-specific reflection patterns, and step boundaries beyond the main extraction rule. Extending RUAR to such richer reflective-step definitions is an important direction for future work.

RUAR also introduces additional RL post-training cost because step utility is estimated with answer-forcing probes. Although these probes are used only during post-training and add no inference-time overhead, the current implementation still requires multiple sampled final-answer continuations for each selected reflective reasoning step. Future work should develop more compute-efficient utility estimation, such as adaptive probe allocation, cheaper surrogate utility predictors, or improved probe reuse across nearby step boundaries.

References

- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. [Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand. Association for Computational Linguistics.
- Daman Arora and Andrea Zanette. 2026. [Training language models to reason efficiently](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Simon A. Aytes, Jinheon Baek, and Sung Ju Hwang. 2025. [Sketch-of-thought: Efficient LLM reasoning with adaptive cognitive-inspired sketching](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24296–24320, Suzhou, China. Association for Computational Linguistics.
- Weize Chen, Jiarui Yuan, Tailin Jin, Ning Ding, Huimin Chen, Zhiyuan Liu, and Maosong Sun. 2026. [The overthinker’s DIET: Cutting token calories with Difficulty-aware training](#). In *The Thirty-ninth An-*

nual Conference on Neural Information Processing Systems.

- Xiaoxue Cheng, Junyi Li, Zhenduo Zhang, Xinyu Tang, Xin Zhao, Xinyu Kong, and Zhiqiang Zhang. 2026. [Incentivizing dual process thinking for efficient large language model reasoning](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Daewon Choi, Jimin Lee, Jihoon Tack, Woomin Song, Saket Dingliwal, Sai Muralidhar Jayanthi, Bhavana Ganesh, Jinwoo Shin, Aram Galstyan, and Sravan Babu Bodapati. 2025. [Think clearly: Improving reasoning via redundant token pruning](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 21437–21451, Suzhou, China. Association for Computational Linguistics.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. [Think you have solved question answering? try arc, the ai2 reasoning challenge](#). *Preprint*, arXiv:1803.05457.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training verifiers to solve math word problems](#). *Preprint*, arXiv:2110.14168.
- Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, and 6 others. 2026. [Process reinforcement through implicit rewards](#).
- Chongyu Fan, Yihua Zhang, Jinghan Jia, Alfred O. Hero, and Sijia Liu. 2026. [Cyclicreflex: Improving reasoning models via cyclical reflection token scheduling](#). In *The Fourteenth International Conference on Learning Representations*.
- Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. [Efficient reasoning models: A survey](#). *Transactions on Machine Learning Research*.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. [CRITIC: Large language models can self-correct with tool-interactive critiquing](#). In *The Twelfth International Conference on Learning Representations*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025. [DeepSeek-R1 incentivizes reasoning in llms through reinforcement learning](#). *Nature*, 645(8081):633–638.
- Ruixin Hong, Hongming Zhang, Xinyu Pang, Dong Yu, and Changshui Zhang. 2024. [A closer look at the self-verification abilities of large language models](#)

in logical reasoning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 900–925, Mexico City, Mexico. Association for Computational Linguistics.

Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. 2026. [Thinkprune: Pruning long chain-of-thought of LLMs via reinforcement learning](#). *Transactions on Machine Learning Research*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. [Large language models cannot self-correct reasoning yet](#). In *The Twelfth International Conference on Learning Representations*.

Maxwell Jia. 2024. AIME 2024 dataset. https://huggingface.co/datasets/Maxwell-Jia/AIME_2024. Dataset.

Dongwei Jiang, Jingyu Zhang, Orion Weller, Nathaniel Weir, Benjamin Van Durme, and Daniel Khashabi. 2024. [Self-\[in\]correct: LLMs struggle with discriminating self-generated responses](#). *Preprint*, arXiv:2404.04298.

Liwei Kang, Yue Deng, Yao Xiao, Zhanfeng Mo, Wee Sun Lee, and Lidong Bing. 2026. [First try matters: Revisiting the role of reflection in reasoning models](#).

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. NuminaMath. <https://huggingface.co/datasets/AI-MO/NuminaMath-CoT>.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. [Let’s verify step by step](#). In *The Twelfth International Conference on Learning Representations*.

Weizhe Lin, Xing Li, Zhiyuan Yang, Xiaojin Fu, Hui-Ling Zhen, Yaoyuan Wang, Xianzhi Yu, Wulong Liu, Xiaosong Li, and Mingxuan Yuan. 2026. [Trimr: Verifier-based training-free thinking trimming for efficient test-time scaling](#). In *The Fourteenth International Conference on Learning Representations*.

Wei Liu, Ruochen Zhou, Yiyun Deng, Yuzhen Huang, Junteng Liu, Yuntian Deng, Yizhe Zhang, and Junxian He. 2026. [Learn to reason efficiently with adaptive length-based reward shaping](#). In *The Fourteenth International Conference on Learning Representations*.

Quanyu Long, Kai Jie Jiang, Jianda Chen, Xu Guo, Leilei Gan, and Wenya Wang. 2026. [Self-verification dilemma: Experience-driven suppression](#)

of overused checking in llm reasoning. *Preprint*, arXiv:2602.03485.

Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. 2025. [O1-Pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning](#). In *2nd AI for Math Workshop @ ICML 2025*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.

MathArena. 2025. HMMT February 2025. https://huggingface.co/datasets/MathArena/hmmt_feb_2025. Dataset.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. [GPQA: A graduate-level google-proof question answering benchmark](#). *Preprint*, arXiv:2311.12022.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. [Proximal policy optimization algorithms](#). *Preprint*, arXiv:1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint*, arXiv:2402.03300.

Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, Zhaoxiang Liu, and Shiguo Lian. 2025. [DAST: Difficulty-adaptive slow-thinking for large reasoning models](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 2322–2331, Suzhou (China). Association for Computational Linguistics.

Dachuan Shi, Abdelkadir Asi, Keying Li, Xiangchi Yuan, Leyan Pan, Wenke Lee, and Wen Xiao. 2026. [Swireasoning: Switch-thinking in latent and explicit for pareto-superior reasoning LLMs](#). In *The Fourteenth International Conference on Learning Representations*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. [Reflexion: language agents with verbal reinforcement learning](#). In *Thirty-seventh Conference on Neural Information Processing Systems*.

Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. [Scaling LLM test-time compute](#)

optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*.

Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota. Association for Computational Linguistics.

Chenlong Wang, Yuaning Feng, Dongping Chen, Zhaoyang Chu, Ranjay Krishna, and Tianyi Zhou. 2025. Wait, we don’t need to “wait”! removing thinking tokens improves reasoning efficiency. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 7459–7482, Suzhou, China. Association for Computational Linguistics.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, Bangkok, Thailand. Association for Computational Linguistics.

Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. 2025. TokenSkip: Controllable chain-of-thought compression in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3351–3363, Suzhou, China. Association for Computational Linguistics.

Violet Xiang, Chase Blagden, Rafael Rafailov, Nathan Lile, Sang T. Truong, Chelsea Finn, and Nick Haber. 2026. Just enough thinking: Efficient reasoning with adaptive length penalties reinforcement learning.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. 2025. SoftCoT: Soft chain-of-thought for efficient reasoning with LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23336–23351, Vienna, Austria. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. Preprint, arXiv:2505.09388.

Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. 2026. Dynamic early exit in reasoning models. In *The Fourteenth International Conference on Learning Representations*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gao-hong Liu, Juncai Liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, and 17 others. 2026. DAPO: An open-source LLM reinforcement learning system at scale. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. 2025. Free process rewards without process labels. In *Forty-second International Conference on Machine Learning*.

Qingcheng Zeng, Weihao Xuan, Leyang Cui, and Rob Voigt. 2025. Thinking out loud: Do reasoning models know when they’re right? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1394–1407, Suzhou, China. Association for Computational Linguistics.

Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. 2025a. Reasoning models know when they’re right: Probing hidden states for self-verification. In *Second Conference on Language Modeling*.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025b. AdaptThink: Reasoning models can learn when to think. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3716–3730, Suzhou, China. Association for Computational Linguistics.

Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. 2024. Small language models need strong verifiers to self-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15637–15653, Bangkok, Thailand. Association for Computational Linguistics.

Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, and Xin Eric Wang. 2026. Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Haoran Zhao, Yuchen Yan, Yongliang Shen, Haolei Xu, Wenqi Zhang, Kaitao Song, Jian Shao, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2026. Let LRMs break free from overthinking via self-braking tuning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Xutong Zhao, Tengyu Xu, Xuwei Wang, Zhengxing Chen, Di Jin, Liang Tan, Yen-Ting Lin, Zishun Yu,

Zhuokai Zhao, Yun He, Sinong Wang, Han Fang, Sarath Chandar, and Chen Zhu. 2025. [Boosting LLM reasoning via spontaneous self-correction](#). In *Second Conference on Language Modeling*.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. 2025a. [Group sequence policy optimization](#). *Preprint*, arXiv:2507.18071.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025b. [ProcessBench: Identifying process errors in mathematical reasoning](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1009–1024, Vienna, Austria. Association for Computational Linguistics.

A RUAR Algorithm

Algorithm 1 summarizes the full RUAR training procedure.

B Training and Baseline Matching Details

Training configuration. Table 5 summarizes the core online RL hyperparameters used for a fair comparison under the same training-step budget. TLMRE, PRIME, and RUAR use the same initialization, Numina-3K data, rollout count, batch size, update budget, optimizer settings, sequence-length limits, and checkpoint-selection rule for each backbone. We do not apply an additional KL penalty.

Table 5: Core training hyperparameters for the training-step-matched comparisons.

Item	Value
Batch size	8 prompts
Rollouts per prompt	16
Advantage estimator	RLOO
Learning rate	2×10^{-6}
PPO clip coefficient	0.2
Rollout sampling	Temperature 1.0, top- p 1.0
Maximum response length	16384 tokens

RUAR configuration. Table 6 reports the scalar hyperparameters used by RUAR. The reflective-step extraction rule, utility estimate, answer-ready step definition, and advantage-rescaling equations are defined in Section 3.

Table 6: RUAR hyperparameters.

Item	Value
Length penalty coefficient λ_{len}	0.2
Answer-forcing samples K	4
Answer-ready threshold τ	0.75
Answer-ready streak length c	3
Post-ready positive multiplier γ_{post}^+	0.25
Post-ready negative multiplier γ_{post}^-	1.25

Training compute. The 1.5B training runs are conducted on two NVIDIA RTX PRO 6000 GPUs, while the Qwen3-8B training runs are conducted on two NVIDIA H200 GPUs.

Baseline implementation. We instantiate TLMRE (Arora and Zanette, 2026) and PRIME (Cui et al., 2026) from their publicly released implementations. TLMRE is implemented as the length-regularized RLOO trainer without reflective-step utility rescaling. PRIME is trained as the implicit process-reward baseline under the

same rollout-generation protocol. Baseline-specific hyperparameters with no counterpart in RUAR are kept at the released-code defaults.

C Evaluation and Metric Details

Evaluation protocol. Table 7 summarizes the decoding hyperparameters used for the main evaluation. All methods use the same single-sample greedy protocol to isolate policy differences without additional sampling variance.

Table 7: Main evaluation decoding hyperparameters.

Item	Value
Number of samples	1
Temperature	0.0
Top- p	1.0
Maximum response length	32768 tokens

Evaluation compute. The 1.5B evaluations are conducted on one NVIDIA RTX PRO 6000 GPU, while the Qwen3-8B evaluations are conducted on one NVIDIA H200 GPU.

Accuracy-efficiency score. We report the accuracy-efficiency score (AES) following O1-Pruner (Luo et al., 2025). Let

$$\Delta_{\text{len}} = \frac{\text{Len}_{\text{base}} - \text{Len}_{\text{model}}}{\text{Len}_{\text{base}}},$$

$$\Delta_{\text{acc}} = \frac{\text{Acc}_{\text{model}} - \text{Acc}_{\text{base}}}{\text{Acc}_{\text{base}}}.$$

Then

$$\text{AES} = \begin{cases} \Delta_{\text{len}} + 3|\Delta_{\text{acc}}|, & \Delta_{\text{acc}} \geq 0, \\ \Delta_{\text{len}} - 5|\Delta_{\text{acc}}|, & \Delta_{\text{acc}} < 0. \end{cases}$$

Higher AES indicates a better accuracy-efficiency trade-off relative to the base model.

D Training-Evaluation Problem Overlap Check

To check whether the Numina-3K training split directly overlaps with the evaluation benchmarks, we compare user-facing problem statements only, excluding system prompts, solutions, and model outputs. We normalize text with Unicode NFKC normalization, lowercasing, whitespace normalization, and removal of punctuation and LaTeX command names. We check normalized exact matches and use word 5-gram Jaccard similarity to detect

Algorithm 1 RUAR: Utility-Guided Advantage Rescaling

Require: Prompts $\mathcal{X}$, policy π_θ , rule-based final-answer verifier **correct**; rollout count N , probe count K ; answer-forcing prompt $\mathcal{A}$; ready threshold τ , consecutive count c ; post-ready scales γ_{post}^+ , γ_{post}^- ; length coefficient λ_{len} .

```
1: for each prompt  $x \in \mathcal{X}$  do
2:   Sample  $N$  rollouts  $\{\mathbf{y}^{(i)}\}_{i=1}^N \sim \pi_\theta(\cdot | x)$ 
3:   Score final answers:  $z_i \leftarrow \text{correct}(x, \mathbf{y}^{(i)})$ 
4:   Compute length-regularized rewards  $R_i \leftarrow z_i - \lambda_{\text{len}} \rho_i$ 
5:   Compute base RLOO advantage signals  $A_{i,t}$ 
6:   for each rollout  $\mathbf{y}^{(i)}$  do
7:     Initialize  $\gamma_{i,t} \leftarrow 1$  for all response tokens  $t$ 
8:     Extract reflective reasoning steps  $\mathcal{S}(\mathbf{y}^{(i)}) = (\mathbf{s}_{i,1}, \dots, \mathbf{s}_{i,m_i})$ 
9:     Set  $b_i \leftarrow m_i + 1$ 
10:    for  $j = 1, \dots, m_i$  do
11:      Set  $\mathbf{s}_{i,j}^- \leftarrow \mathbf{y}_{1:a_{i,j}-1}^{(i)}$ 
12:      Estimate  $p_{i,j}^-$  from  $K$  samples of  $\pi_\theta(\cdot | x, \mathbf{s}_{i,j}^- \oplus \mathcal{A})$  using correct
13:      if  $j \geq c$  and  $p_{i,r}^- \geq \tau$  for all  $r = j - c + 1, \dots, j$  then
14:        Set  $b_i \leftarrow j - c + 1$  and stop probing later steps
15:      break
16:    end if
17:  end for
18:  for each pre-ready step  $\mathbf{s}_{i,j}$  with  $j < b_i$  do
19:    Set  $\mathbf{s}_{i,j}^+ \leftarrow \mathbf{y}_{1:e_{i,j}}^{(i)}$ 
20:    Estimate  $p_{i,j}^+$  from  $K$  samples of  $\pi_\theta(\cdot | x, \mathbf{s}_{i,j}^+ \oplus \mathcal{A})$  using correct
21:    Set utility  $u_{i,j} \leftarrow p_{i,j}^+ - p_{i,j}^-$ 
22:    for each token  $t \in \mathbf{s}_{i,j}$  do
23:       $\gamma_{i,t} \leftarrow \max(0, 1 + u_{i,j})$  if  $A_{i,t} \geq 0$ , else  $\max(0, 1 - u_{i,j})$ 
24:    end for
25:  end for
26:  if  $b_i \leq m_i$  then
27:    for each post-ready reasoning token  $t$  do
28:       $\gamma_{i,t} \leftarrow \gamma_{\text{post}}^+$  if  $A_{i,t} \geq 0$ , else  $\gamma_{\text{post}}^-$ 
29:    end for
30:  end if
31:  Set  $A'_{i,t} \leftarrow \gamma_{i,t} A_{i,t}$  for all response tokens  $t$ 
32: end for
33: end for
34: Apply standard advantage normalization to  $A'_{i,t}$ 
35: Update  $\pi_\theta$  using the PPO-style objective with the normalized RUAR-adjusted advantage signal
```

near duplicates. An evaluation problem is flagged as a near duplicate if its maximum similarity to any Numina-3K training problem is at least 0.8.

Benchmark	Rows	Near-dup. ≥ 0.8	Max sim.
GSM8K	1319	0	0.043
MATH500	500	0	0.643
AIME2024	30	0	0.121
HMMT2025	30	0	0.143
GPQA	198	0	0.061
ARC-C	1172	0	0.100
CSQA	1221	0	0.015

Table 8: Problem-statement overlap check between the Numina-3K training split and the evaluation benchmarks.

The exact-match check finds no direct problem-statement overlap. As shown in Table 8, no evaluation problem is flagged as a near-duplicate above the 0.8 Jaccard threshold. The largest observed similarity is 0.643, from a MATH500 conic-classification problem and a Numina-3K training problem with the same answer-choice template but a different equation. These results suggest that the reported gains are not explained by direct problem-statement memorization from Numina-3K.

E Additional Training Analysis

E.1 Training Dynamics

Table 9 analyzes how RUAR evolves during training on the Numina validation split. Validation accuracy improves quickly and peaks at step 150, while the response length is already substantially reduced. After this point, later checkpoints continue to shorten responses and reduce explicit *Wait/Alternatively* reflective tokens, but accuracy starts to decline. This suggests that further training can lead to over-compression: reducing reflective continuation is useful only up to the point where necessary correction or verification steps are still preserved. Therefore, we select checkpoints by the validation accuracy-length trade-off, rather than by minimum length or minimum reflective-token frequency.

E.2 Training-Time Cost Analysis

Probe overhead. We report per-step wall-clock time averaged over training steps 1–150, using the same matched training setup as the main experiments: batch size 8 prompts and $N = 16$ rollouts per prompt. RUAR adds a training-only answer-forcing probe stage: for selected reflective-step

Training Step	Acc.	Avg. Tok.	Wait+Alt./resp.
50	38.1	3273	8.55
100	41.8	2742	6.35
150	43.8	2275	5.03
200	43.6	1519	3.56
250	41.4	1290	2.99
300	36.1	1254	2.08

Table 9: Training dynamics on the 1.5B model using the 512-problem Numina validation split.

boundaries, it samples $K = 4$ short final-answer continuations and scores them with the rule-based final-answer verifier. The reported probe time is the aggregate per-step cost of this batched probe stage, not the cost of a single verifier call. To reduce redundant probes, we deduplicate identical adjacent probe points and reuse each after-step estimate as the next before-step estimate when scanning consecutive reflective reasoning steps.

Method	Total	Rollout	Probe	Other
TLMRE	269	175	0	94
RUAR	556	166	301	89

Table 10: Per-step training time in seconds on the 1.5B setting, averaged over steps 1–150 excluding validation. Total is wall-clock time; Rollout is initial RLOO generation; Probe is batched answer-forcing; Other mainly includes policy scoring and actor update.

At inference time, RUAR uses the trained policy directly and does not require answer-forcing probes, a learned reward model, or additional verifier calls.

F Length-Regularized Terminal Reward

For a fixed prompt x , let $L_i = |\mathbf{y}^{(i)}|$ be the generated response length and let $z_i = z(x, \mathbf{y}^{(i)})$ denote final-answer correctness. Following [Arora and Zanette \(2026\)](#), we use the terminal reward

$$R_i = z_i - \lambda_{\text{len}} \rho_i,$$

where ρ_i is a response-level length penalty and λ_{len} controls its strength.

The penalty is applied only to correct rollouts. This encourages efficiency among successful solutions without rewarding short incorrect attempts. Among correct rollouts for the same prompt, let μ_L and σ_L denote the mean and standard deviation of their lengths. We compute

$$\rho_i = z_i \cdot \text{sigmoid} \left(\frac{L_i - \mu_L}{\max(\sigma_L, \epsilon)} \right),$$

where ϵ is a small constant used only for numerical stability. If a prompt has no correct rollout, the length penalty is set to zero for all rollouts of that prompt.

Since ρ_i is multiplied by z_i , incorrect rollouts receive $\rho_i = 0$ and therefore $R_i = 0$. For correct rollouts, the reward remains near one but is reduced more for responses that are longer than other correct rollouts for the same prompt.

G Answer-Forcing Prompt and Verification Details

This appendix describes how answer-forcing probes are generated and verified. For a probe point corresponding to a before-step or after-step partial trace $y_{1:q}$, we append the answer-forcing prompt

$$\mathcal{A} = \backslash\text{**Final Answer**}\backslash\text{boxed}.$$

The probe input to the policy is the original prompt followed by the partial trace and $\mathcal{A}$. In our experiments, we use $K = 4$ and sample short final-answer continuations from the current policy, with each continuation capped at 32 new tokens.

For verification, each sampled continuation is concatenated back to the probed partial trace, yielding a completion of the form

$$y_{1:q} \oplus \mathcal{A} \oplus o_k.$$

This completion is passed to the same rule-based final-answer scorer used for terminal outcome scoring, together with the gold answer and data-source metadata. The scorer extracts a candidate final answer from the completion and compares it against the gold answer. In our math setting, the extractor prioritizes the final boxed answer when present and otherwise falls back to standard answer markers; the extracted candidate is then normalized and checked for mathematical equivalence with the gold answer. The verifier returns 1 if the extracted answer is judged equivalent to the gold answer and 0 otherwise.

For each probe point, the correct-answer probability is estimated by averaging the K binary verification results. Thus, p_j^- and p_j^+ are Monte Carlo estimates of whether the before-step and after-step partial traces can already lead to the correct final answer under the answer-forcing prompt. These probe continuations are used only for estimating the step utility $u_j = p_j^+ - p_j^-$; they are not treated as additional RL rollouts and do not provide process labels or learned process rewards.

H Method-Aligned Reflective-Token Inventory

To quantify reflective-token usage under the same extraction convention used by RUAR, we apply the method-level reflective-step extractor to DeepSeek-R1-Distill-Qwen-1.5B MATH500 base traces. Unlike the broader line-initial marker inventory, this diagnostic uses only exact case-sensitive *Wait* and *Alternatively* tokens inside the thinking trace. The main RUAR setting uses *Wait* as the target step delimiter, while both *Wait* and *Alternatively* are tracked as endpoint tokens.

Reflective tok.	Count	Share	Per resp.	Resp. w/ token
<i>Wait</i>	7,036	84.8	14.07	477/500
<i>Alternatively</i>	1,257	15.2	2.51	284/500
Total	8,293	100.0	16.59	479/500

Table 11: Method-aligned reflective-token inventory on DeepSeek-R1-Distill-Qwen-1.5B MATH500 base traces. Counts use the same exact case-sensitive *Wait/Alternatively* tracking convention as RUAR, without line-initial restrictions.

The distribution is highly concentrated: *Wait* accounts for 84.8% of the tracked reflective tokens and appears in 95.4% of responses. This supports using *Wait* as the main target delimiter for reflective reasoning steps. At the same time, *Alternatively* appears frequently enough to serve as an endpoint token, preventing distinct reflective transitions from being merged into a single credit-assignment unit.

I Answer-Ready Step Analysis

Answer-ready step detection during training. RUAR uses the answer-ready step to identify when a before-step partial trace is already likely to produce the correct final answer under answer-forcing. Table 12 summarizes how often such a step is detected during 1.5B training. Each training batch contains 8 prompts and 16 rollouts per prompt, yielding 128 response rollouts.

The answer-ready step is detected sparsely but consistently: across all logged steps, it appears in 6.1% of responses on average, or 7.8 responses per 128-response batch. The detection rate is highest early in training, when responses are longest, and declines as the policy becomes shorter. This is consistent with its role in RUAR: once the model reaches an answer-ready step, subsequent reflective continuation is treated as post-ready continuation and receives fixed rescaling. As training progresses,

Steps	Detect.	Count@128	Avg. Tok.
All	6.1	7.8	2733
1–50	18.1	23.2	5590
51–100	5.9	7.5	3652
101–150	4.7	6.0	2708
151–200	3.5	4.4	1932
201–250	3.2	4.1	1543
251–299	1.0	1.3	936

Table 12: Answer-ready step detection during 1.5B RUAR training. Detect. is the percentage of response rollouts with a detected answer-ready step. Count@128 is the average number of detected response rollouts per 128-response training batch, corresponding to 8 prompts $\times$ 16 rollouts.

the policy gradually produces fewer such intervention points.

J Component Ablation

Table 13 ablates RUAR’s training signals on the 512-problem Numina-3K validation split. Plain RLOO provides little improvement over the base model, indicating that online RL alone does not explain the gains. The length-only variant corresponds to TLMRE, using the length penalty without reflection-utility rescaling; it produces the shortest responses but does not achieve the best accuracy. Reflection-utility rescaling alone also substantially improves accuracy over the base model, showing that the step-level utility signal is useful even without length pressure. Full RUAR achieves the highest accuracy while still reducing average response length substantially, suggesting that reflection-step utility complements the length objective rather than simply maximizing compression.

Variant	RLOO	Len.	Refl. util.	Acc.	Avg. Tok.
Base	–	–	–	28.9	4124
RLOO only	✓	–	–	31.1	4003
Length only	✓	✓	–	40.0	1646
Reflection only	✓	–	✓	41.2	2899
Full RUAR	✓	✓	✓	43.8	2275

Table 13: Component ablation on the Numina validation split. Except for Base, all variants are evaluated at step 150. Len. denotes the length penalty, and Refl. util. denotes reflection-step utility advantage rescaling.

K Additional Sensitivity Analyses

Unless otherwise stated, all sensitivity analyses use the 1.5B backbone, step-50 checkpoints, greedy single-sample decoding on MATH500, and a 16384-token response limit. Acc. and 16k clip

are reported in percent, where 16k clip denotes the fraction of responses reaching the response-length limit.

K.1 Length-Penalty Coefficient Sensitivity

We evaluate the sensitivity of RUAR to the length-penalty coefficient λ_{len} . The $\lambda_{\text{len}} = 0.0$ row disables only the response-level length penalty while keeping reflective-step utility rescaling enabled, so it serves as a no-length-penalty proxy for this sweep.

λ_{len}	Acc.	Correct	Avg. Tok.	16k clip
0.0	71.0	355/500	2700	1.0
0.1	76.2	381/500	2557	1.2
0.2	79.2	396/500	2221	0.4
0.3	75.4	377/500	2053	1.0

Table 14: Length-penalty coefficient sensitivity on MATH500.

The best MATH500 accuracy is obtained at $\lambda_{\text{len}} = 0.2$, which is the coefficient used in RUAR. Increasing the coefficient to 0.3 produces shorter responses but lower accuracy, suggesting over-compression on this external benchmark.

K.2 Reflective-Token Sensitivity

We evaluate whether RUAR’s reflective-step extraction rule depends strongly on the choice of reflective token. Our main setting uses exact case-sensitive *Wait* tokens as target step-start delimiters, and ends each step at the next tracked reflective token, either *Wait* or *Alternatively*. We compare this setting against variants that change either the target step-start token or the endpoint token.

Target start	Endpoint	Acc.	Correct	Avg. Tok.
<i>Wait</i>	<i>Wait/Alt.</i>	79.2	396/500	2221
<i>Alternatively</i>	<i>Wait/Alt.</i>	75.4	377/500	2248
<i>Wait/Alt.</i>	<i>Wait/Alt.</i>	75.8	379/500	2229
<i>Wait</i>	<i>Wait</i>	77.0	385/500	2190

Table 15: Reflective-token sensitivity on MATH500. Target start denotes the token used to start target reflective reasoning steps, and Endpoint denotes the tracked token used to end the current step.

Our main *Wait*-target setting with both *Wait* and *Alternatively* as endpoint tokens achieves the highest MATH500 accuracy. Using *Alternatively* as the target start token, or treating both *Wait* and *Alternatively* as target start tokens, reduces accuracy while producing similar response lengths. Restricting the

endpoint token to *Wait* alone gives slightly shorter responses but also lower accuracy than the main setting. These results suggest that *Wait* tokens provide a useful empirical anchor for reflection-step probing, while allowing *Alternatively* to terminate steps helps avoid merging distinct reflective transitions into a single credit-assignment unit.

K.3 Answer-Ready Step Threshold Sensitivity

We evaluate the sensitivity of RUAR to the answer-ready threshold τ , changing only τ while keeping all other settings fixed.

Threshold τ	Acc.	Correct	Avg. Tok.	16k clip
0.25	74.4	372/500	2062	0.8
0.50	77.8	389/500	2209	0.8
0.75	79.2	396/500	2222	0.4
1.00	75.6	378/500	2189	0.6

Table 16: Answer-ready threshold τ sensitivity on MATH500.

The default threshold $\tau = 0.75$ achieves the best accuracy. Lowering the threshold to $\tau = 0.25$ shortens responses but substantially reduces accuracy, suggesting that the answer-ready step can be detected too early and suppress useful continuation. Increasing the threshold to $\tau = 1.0$ also lowers accuracy, suggesting that overly conservative answer-ready step detection weakens post-ready scaling.

L Utility-Composition Diagnostic Details

For the utility-composition analysis in Table 2, we apply the same answer-forcing probe procedure as in Section 3.2. For each extracted target reflective reasoning step s_j , we estimate p_j^- and p_j^+ from the before-step and after-step partial traces, respectively. During training, RUAR uses the utility estimate $u_j = p_j^+ - p_j^-$ directly for advantage rescaling. In this diagnostic analysis, we additionally assign threshold-based labels to summarize how each step changes the answer-readiness status of the partial trace. We classify each step by thresholding both estimates at the answer-ready threshold $\tau = 0.75$:

	$p_j^+ < \tau$	$p_j^+ \geq \tau$
$p_j^- < \tau$	<i>Ineffective</i>	<i>Helpful</i>
$p_j^- \geq \tau$	<i>Harmful</i>	<i>Redundant</i>

Thus, steps labeled as helpful move a partial trace from not answer-ready to answer-ready, redundant steps remain answer-ready before and after the

step, ineffective steps remain not answer-ready, and harmful steps move a partial trace from answer-ready to not answer-ready.

All extracted target reflective reasoning steps are probed in this diagnostic; no answer-ready early stopping is applied during probing. Step endpoints follow the main extraction rule: each target step ends immediately before the next tracked reflective token, either *Wait* or *Alternatively*, or at the end of the reasoning segment if no later tracked token appears.

Table 17 reports response lengths, raw reflective-token counts, and extracted step counts. In the main extraction rule, *Wait* defines target reflective reasoning steps, while *Alternatively* is used as an endpoint token. We therefore report raw *Wait* and *Alternatively* counts separately; utility percentages are computed only over extracted target reflective reasoning steps.

Method	Avg. Tok.	#Wait _{raw}	#Alt. _{raw}	#Steps
Base	5342	8594	4720	8590
PRIME	10184	11391	10302	11380
TLMRE	3414	3588	1080	3586
RUAR	2564	2574	1061	2572

Table 17: Response length, raw reflective-token counts, and extracted step counts on Qwen3-8B MATH500. #Wait_{raw} and #Alt._{raw} count case-sensitive raw occurrences of *Wait* and *Alternatively* over all 500 responses. #Steps denotes the number of extracted target reflective reasoning steps probed for utility.

Table 18 reports the corresponding full utility composition. Compared with the base model, RUAR reduces the absolute number of harmful steps from 75 to 29, a 61.3% reduction. Compared with TLMRE, RUAR also reduces harmful steps from 42 to 29, a 31.0% reduction. Although harmful steps are rare for all methods and their percentages should be interpreted together with the total number of extracted steps, this result suggests that RUAR reduces low-utility reflection without increasing harmful derailment.

M Pre/Post-Ready Utility Breakdown

To complement the post-ready continuation analysis in Table 3, we further decompose extracted target reflective reasoning steps by the diagnostic utility category introduced in Appendix L, before and after the detected answer-ready step. For a response with answer-ready step index b , steps with $j < b$ are counted as pre-ready, while steps with

Method	Helpful	Redundant	Ineffective	Harmful
Base	273 (3.18)	5328 (62.03)	2914 (33.92)	75 (0.87)
PRIME	333 (2.93)	7735 (67.97)	3188 (28.01)	124 (1.09)
TLMRE	146 (4.07)	1320 (36.81)	2078 (57.95)	42 (1.17)
RUAR	158 (6.14)	1026 (39.89)	1359 (52.84)	29 (1.13)

Table 18: Full utility composition of extracted target reflective reasoning steps on Qwen3-8B MATH500. Percentages are computed over extracted target reflective reasoning steps for each method.

$j \geq b$ are counted as post-ready. If no answer-ready step is detected, all extracted target reflective reasoning steps are counted as pre-ready and there is no post-ready continuation.

We use the same answer-forcing diagnostic as in Appendix L. These utility categories are used only as diagnostic labels for analysis; during training, RUAR uses the utility estimate $u_j = p_j^+ - p_j^-$ directly for advantage rescaling. Percentages in Table 19 are computed within each method–region pair. The post-ready rows are dominated by low-utility steps across methods, supporting the interpretation that post-ready continuation is mostly confirmatory or ineffective rather than corrective. RUAR reduces the number of post-ready target reflective reasoning steps from 5390 in Base and 1364 in TLMRE to 805, while most remaining post-ready steps are still low-utility.

Method	Region	#Steps	Helpful	Low-util.
Base	Pre-ready	3200	237 (7.41)	2963 (92.59)
Base	Post-ready	5390	36 (0.67)	5354 (99.33)
PRIME	Pre-ready	3147	263 (8.36)	2884 (91.64)
PRIME	Post-ready	8233	70 (0.85)	8163 (99.15)
TLMRE	Pre-ready	2222	123 (5.54)	2099 (94.46)
TLMRE	Post-ready	1364	23 (1.69)	1341 (98.31)
RUAR	Pre-ready	1767	148 (8.38)	1619 (91.62)
RUAR	Post-ready	805	10 (1.24)	795 (98.76)

Table 19: Pre/post-ready utility composition of extracted target reflective reasoning steps on Qwen3-8B MATH500. Percentages are computed within each method–region pair. Low-util. combines redundant, ineffective, and harmful steps.

N Case Study: Endpoint Delimitation by *Alternatively*

We provide an extraction-level case study illustrating how RUAR uses both *Wait* and *Alternatively* to determine step endpoints, while using only *Wait* as the target step-start delimiter in the main setting. The example is taken from a Qwen3-8B GSM8K base trace (gsm8k-test-218). The model has al-

ready derived the correct answer [45] and then continues with reflective verification. Table 20 shows how this continuation is segmented under the main extraction rule, including how an *Alternatively*-start continuation closes the preceding *Wait*-start target reflective reasoning step without itself becoming a target step.

O Case Study: Reflection Step Utility and Answer-Ready Step

We provide a method-level case study of RUAR on a real MATH500 rollout from the Qwen3-8B step-probe diagnostic, using MATH500 sample index 220. The completed rollout is correct but long: it contains 14612 response tokens and 41 *Wait* reflective tokens. For each extracted target reflective reasoning step, we probe its before-step and after-step partial traces. Concretely, for a partial trace h , we append the fixed answer-forcing prompt `\n**Final Answer**\n\boxed` and sample $K = 4$ short final-answer continuations from the current policy, with a maximum of 32 new tokens. This prompt does not reveal the gold answer; it only forces the model to stop reasoning and complete a boxed final answer. Each completed probe is scored by the rule-based final-answer verifier against the gold answer, and the correct-answer probability estimate is the fraction of the four probes that are correct.

Since this example illustrates a correct high-reward trajectory, the table describes the scaling applied to positive-advantage tokens; the symmetric negative-advantage case is defined in Section 3.3. After three consecutive near answer-ready steps with $p^- \geq \tau = 0.75$, RUAR identifies the first step in that consecutive run as the answer-ready step and applies fixed post-ready rescaling from that step onward within the reasoning segment y_r . Table 21 summarizes how RUAR assigns pre-ready step utilities, identifies the answer-ready step, and switches to fixed post-ready rescaling in this rollout.

P Case Study: Reducing Redundant Reflective Continuation

We provide a case study on GSM8K example gsm8k-test-97 using the 1.5B policies, where both Base and RUAR reach the correct answer but differ substantially in how much redundant reflective continuation they generate. The gold answer is [12]. Base uses 2462 response tokens

with 11 extracted target reflective reasoning steps, while RUAR uses 329 response tokens with 1 extracted target reflective reasoning step. The traces initially follow the same solution path, but Base repeatedly reopens the solved reasoning state after *Wait* and temporarily considers a misleading 6-tomato shortcut. In Table 22, the green-highlighted step marks the Base reflective reasoning step that increases correct-answer probability ($p^- = 0.00 \rightarrow p^+ = 1.00$). The final row summarizes the answer-forcing results for all extracted target reflective reasoning steps in this example.

Region	Rollout excerpt	RUAR interpretation
Problem	“Adrian’s age is three times the age of Harriet, and Harriet is half the age of Zack. Calculate the average age of the three in three years if Harriet is 21 years old now.” Gold answer: 45 .	
Solved partial trace	“... In three years, their ages will be 24, 66, and 45. Adding those: $24 + 66 = 90$, and $90 + 45 = 135$. Dividing by 3 gives 45. So, the average age in three years will be 45 years.”	The partial trace has already derived the correct answer. The following continuation contains multiple reflective tokens, making it useful for illustrating how RUAR extracts target reflective reasoning steps.
First <i>Wait</i> -start target step	“ <i>Wait</i> , let me check if I did everything correctly. Let me verify each step again. First, Harriet is 21. Adrian is three times that, so $3 \times 21 = 63$. Zack is twice Harriet’s age, so $2 \times 21 = 42$. In three years, their ages will be 24, 66, and 45. Adding those gives 135. Dividing by 3 gives 45.”	This is a target reflective reasoning step because it starts with <i>Wait</i> . In the main setting, RUAR probes its before-step and after-step partial traces and can assign the step utility $u_j = p_j^+ - p_j^-$.
Endpoint delimiter	“ <i>Alternatively</i> , maybe there’s another way to approach this problem? Sometimes, averages can be calculated by considering the current average and then adding the number of years ...”	This <i>Alternatively</i> token closes the preceding <i>Wait</i> -start target step. It is used to determine the step endpoint, so the previous target step ends immediately before <i>Alternatively</i> . In the main setting, this <i>Alternatively</i> -start continuation is not extracted as a separate target reflective reasoning step.
Next <i>Wait</i> -start target step	“ <i>Wait</i> , the average age in three years would be the current average age plus three years. Because each person’s age increases by three years, the average also increases by three years. The current total age is $21 + 63 + 42 = 126$, so the current average is 42. Then, in three years, the average is $42 + 3 = 45$.”	When the next <i>Wait</i> appears, RUAR extracts a new target reflective reasoning step. This new <i>Wait</i> -start step is again eligible for answer-forcing utility estimation and utility-guided advantage rescaling. Thus, <i>Alternatively</i> can close the previous step without becoming a target step itself.
Resulting step endpoints	Under the main extraction rule, the first target reflective reasoning step is $\mathbf{s}_j = [\textit{Wait}, \dots, \text{immediately before } \textit{Alternatively}),$ and the next target reflective reasoning step starts at the following <i>Wait</i> token: $\mathbf{s}_{j+1} = [\textit{Wait}, \dots).$ Thus, <i>Wait</i> determines where target steps start, while both <i>Wait</i> and <i>Alternatively</i> can determine where the current step ends.	
Scaling consequence	Only extracted <i>Wait</i> -start target reflective reasoning steps receive step utilities in the main setting. The <i>Alternatively</i> -start continuation closes the previous target step but does not receive its own utility value unless the target-token setting is changed. Tokens outside pre-ready target reflective reasoning steps use the default multiplier $\gamma_{i,t} = 1$, unless they fall in post-ready continuation, in which case the fixed post-ready multipliers are applied as defined in Section 3.4.	

Table 20: Example of endpoint delimitation. A target reflective reasoning step starts at an exact *Wait* token and ends immediately before the next tracked reflective token, here *Alternatively*. In the main RUAR setting, *Alternatively* is used to determine step endpoints but is not used as a target step-start delimiter; the next *Wait* starts a new target step.

Stage	Rollout excerpt	p^-	p^+	RUAR action
Problem	“Two numbers, x and y , are selected at random from the interval $(0, 3)$. What is the probability that a triangle with sides of length 1, x , and y exists?” Gold answer: $\boxed{\frac{1}{2}}$.			
Initial partial trace	“... The triangle inequalities are $1 + x > y$, $1 + y > x$, and $x + y > 1$... visualize this as a region in the coordinate plane where $x, y \in (0, 3)$... the total area is $3 \times 3 = 9$...”	–	–	–
Step 1	“ <i>Wait</i> , no. Let me check ... $y = 1 + x$ and $x = 1 + y$, rewritten as $y = x - 1$... so these two lines are parallel?”	0.00	0.00	Neutral step: $u = 0.00$. Positive credit is unchanged ($\gamma = 1 + u = 1.00$).
Step 2	“ <i>Wait</i> , both have slope 1 ... the region between them is bounded by $y = x + 1$ and $y = x - 1$ inside the square ...”	0.00	0.50	$u = +0.50$. This step increases the correct-answer probability, so positive credit in the pre-ready step is amplified by $\gamma = 1 + u = 1.50$.
Step 3	“ <i>Wait</i> , but if we have both $y < 1 + x$ and $x < 1 + y$, then combining these gives $ x - y < 1$... area between these two lines within the square ...”	0.50	0.00	$u = -0.50$. Although the rollout later recovers, this step decreases the correct-answer probability at this partial trace, so positive credit in the step is reduced to $\gamma = 1 + u = 0.50$.
Step 4	“ <i>Wait</i> , actually, the area of this triangle is $(\text{base} \times \text{height})/2$... maybe it is easier to think of it as a right triangle.”	0.00	0.00	Neutral step: $u = 0.00$. The step does not change the local correct-answer probability estimate, so positive credit is left unchanged.
Step 5	“ <i>Wait</i> , when $y \geq x + 1$... the area above this line is a triangle ... area 2 ... similarly, the area where $y \leq x - 1$ is also 2 ... so $ x - y < 1$ has area $9 - 4 = 5$... subtract $x + y \leq 1$, area 0.5 ... probability $4.5/9 = \frac{1}{2}$.”	0.00	1.00	$u = +1.00$. This step makes the partial trace likely to produce the correct answer, so positive credit is strongly amplified ($\gamma = 2.00$).
Ready probe 1	“ <i>Wait</i> , but let me verify this again ... total area where $ x - y < 1$ is 5 ... area where $ x - y < 1$ and $x + y \leq 1$ is 0.5 ... probability is $4.5/9 = \frac{1}{2}$...”	1.00	–	First near answer-ready step. Since $p^- \geq \tau = 0.75$, RUAR starts a consecutive run and records this step as the candidate answer-ready step. If the run reaches length $c = 3$, this candidate is confirmed as the answer-ready step, and this step itself is included in the post-ready region.
Ready probe 2	“ <i>Wait</i> , maybe I can calculate the area directly ... the conditions are $ x - y < 1$ and $x + y > 1$...”	$\geq$ 0.75	–	Second consecutive near answer-ready step. The consecutive run continues; once the run is confirmed, this step is treated as post-ready continuation rather than a pre-ready utility-guided step.
Ready probe 3	“ <i>Wait</i> , perhaps it is a polygon with vertices $(0, 1)$, $(1, 0)$, $(3, 2)$, and $(2, 3)$...”	$\geq$ 0.75	–	Third consecutive near answer-ready step. The candidate from Ready probe 1 is confirmed as the answer-ready step, so Ready probe 1 and all later reasoning-segment tokens receive fixed post-ready scaling, while the answer segment y_a is excluded.
Post-ready continuation	“... Alternatively, maybe I can calculate the area directly ... <i>Wait</i> , perhaps it is a polygon ... <i>Wait</i> , no ... let me go back to the subtraction method ... $9 - 4 = 5$... subtract 0.5 ... probability $\frac{1}{2}$...” Answer segment y_a “... Final answer: $\boxed{\frac{1}{2}}$.”	–	–	All reasoning-segment tokens from the answer-ready step onward are treated as post-ready continuation. For this correct trajectory, positive credit over the gray region is reduced ($\gamma_{\text{post}}^+ = 0.25$), while the answer segment y_a is excluded from fixed post-ready rescaling.

Table 21: Method-level case study from Qwen3-8B on MATH500 sample 220. The same rollout contains helpful, harmful, and answer-ready reflective reasoning steps. Gray text denotes post-ready continuation; the answer segment y_a is excluded from fixed post-ready rescaling.

Stage	Base (2462 tok., 11 target steps)	RUAR (329 tok., 1 target step)
Problem	“Freda is cooking canned tomatoes into sauce. She loses half the volume when cooking. Each 16-ounce can contains three tomatoes. Her last batch made 32 ounces of sauce. How many tomatoes did she use?” Gold answer: 12.	
Problem setup (238 vs. 54 tok.)	“Okay, so Freda is cooking canned tomatoes into sauce, and she loses half the volume when she does that. Each can she uses has three tomatoes, and her last batch made 32 ounces of sauce. I need to figure out how many tomatoes she used. Hmm, let me break this down step by step. First, let’s understand the process. She starts with some number of cans of tomatoes. Each can has three tomatoes. When she cooks the tomatoes into sauce, she loses half the volume ...”	“Okay, let me try to figure out how many tomatoes Freda used. Hmm, so she cooks canned tomatoes into sauce and loses half their volume. Each can has three tomatoes and is 16 ounces. Her last batch made 32 ounces of sauce.”
Correct path reached (green step: Base $p^- = 0.00 \rightarrow p^+ = 1.00$)	“<i>Wait</i>, so if she uses c cans, the total volume of tomatoes is $16c$ ounces. Then, when she cooks them, she loses half the volume, so the sauce made is $(16c)/2 = 8c$ ounces. But she made 32 ounces of sauce. So, $8c = 32$. Therefore, $c = 32/8 = 4$. So, she used 4 cans of tomatoes. Since each can has 3 tomatoes, the total number of tomatoes is $4 \times 3 = 12$.”	“First, I need to find out how much tomato volume she started with before losing half. Since she ended up with 32 ounces after losing half, that means the original volume was double that. So, $32 \times 2 = 64$ ounces. Now, each can is 16 ounces ... $64/16 = 4$ cans. Each can has three tomatoes, so $4 \times 3 = 12$ tomatoes.”
After deriving the answer (1765 vs. 76 tok.)	“<i>Wait</i>, that seems straightforward, but let me double-check She made 32 ounces of sauce. Since the sauce is half the volume of the tomatoes, the total volume of tomatoes she used was $32 \times 2 = 64$ ounces ... $64/16 = 4$ cans ... $4 \times 3 = 12$ tomatoes. Alternatively, maybe thinking about the number of tomatoes per ounce? Each can has 3 tomatoes and 16 ounces, so $3/16$ tomatoes per ounce. Then, 32 ounces would be $32 \times (3/16) = 6$ tomatoes. <i>Wait</i>, that’s different. Hmm, that can’t be right because 6 is not 12. <i>Wait</i>, let me recast that ... But that contradicts the previous answer. So, which one is correct? <i>Wait</i>, maybe I messed up the ratio ... 32 ounces of sauce is 64 ounces of tomatoes ... 4 cans ... 12 tomatoes. ... <i>Wait</i>, but let me think about the initial problem again ... <i>Wait</i>, but let me think about the ratio again ... So, the answer is 12 tomatoes.”	“<i>Wait</i>, let me check that again. Each can is 16 ounces, and she used 4 cans, so $4 \times 16 = 64$ ounces. She started with 64 ounces, lost half, so 32 ounces made. Each can has 3 tomatoes, so $4 \times 3 = 12$ tomatoes. Yeah, that seems right.”
Conclusion (277 vs. 80 tok.)	“Freda used 12 tomatoes. ... Let c be the number of cans used. The total volume of tomatoes is $16c$ ounces. When cooking, the volume of sauce is half of this, so $8c = 32$, hence $c = 4$. Each can contains 3 tomatoes, so $4 \times 3 = 12$. Thus, Freda used 12 tomatoes.”	“Freda used 12 tomatoes. Freda started with 64 ounces of tomato volume, which came from 4 cans. Each can contains 3 tomatoes, so she used $4 \times 3 = 12$ tomatoes. 12”
Diagnostic summary (all target steps)	11 target steps: 1 helpful, 10 redundant, 0 ineffective, 0 harmful. The green-highlighted reflective reasoning step changes correct-answer probability from $p^- = 0.00$ to $p^+ = 1.00$. The remaining 10 target steps are redundant ($p^- = p^+ = 1.00$), showing that Base repeatedly continues after the partial trace is already answer-ready.	1 target step: 0 helpful, 1 redundant, 0 ineffective, 0 harmful. The single target step is a short redundant verification ($p^- = p^+ = 1.00$). Thus, RUAR reduces repeated redundant post-ready continuation from 10 steps to 1 step.

Table 22: Qualitative comparison on GSM8K example gsm8k-test-97. Both methods answer correctly, but Base repeatedly reopens the solved reasoning state after deriving the answer and temporarily considers a misleading 6-tomato shortcut. The green-highlighted step marks the Base reflective reasoning step that increases correct-answer probability ($p^- = 0.00 \rightarrow p^+ = 1.00$); the final row shows that the remaining 10 Base target steps are redundant, while RUAR reduces this repeated redundant continuation to one short verification step.